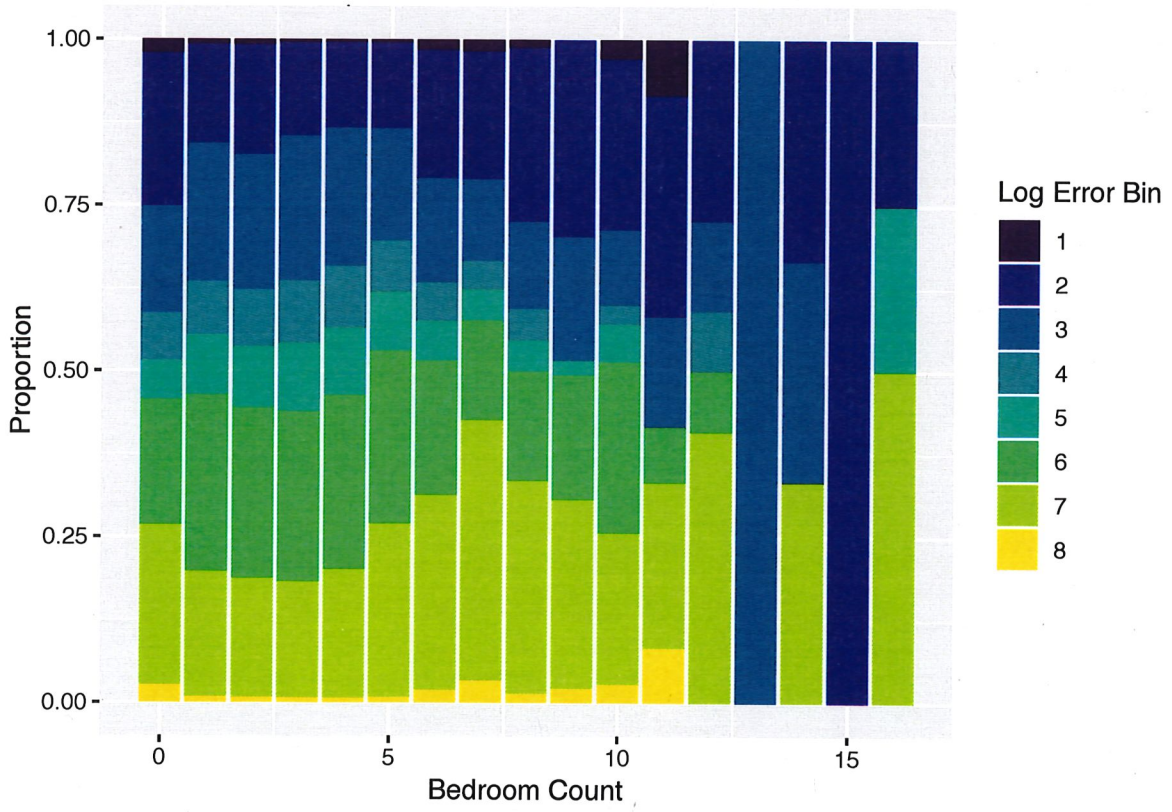
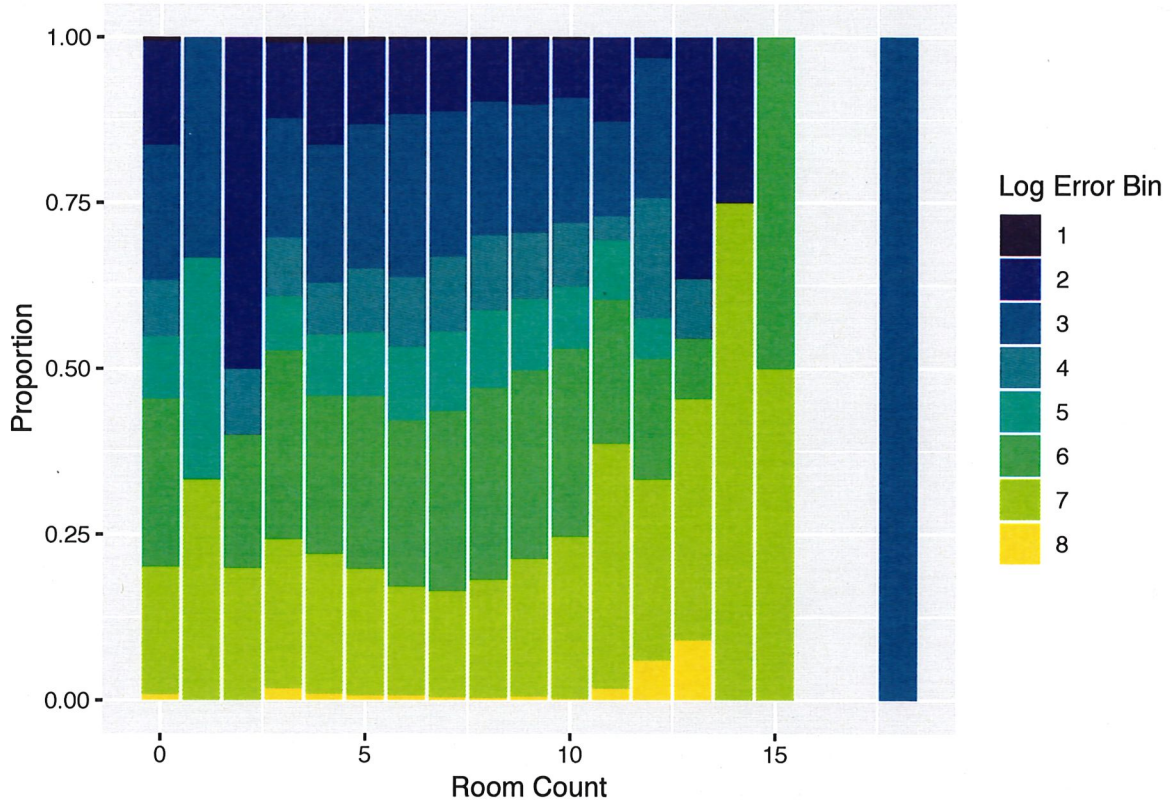
When comparing the response with a categorical variable, or a continuous variable with a finite number of possible values, a bar chart can be useful:

```
> ggplot(trainmerged, aes(x=bedroomcnt, fill=logerrorbin)) +
+   geom_bar(position="fill") +
+   labs(x = "Bedroom Count", y="Proportion",
+        fill="Log Error Bin")
>
> ggplot(trainmerged, aes(x=roomcnt, fill=logerrorbin)) +
+   geom_bar(position="fill") +
+   labs(x = "Room Count", y="Proportion",
+        fill="Log Error Bin")
```

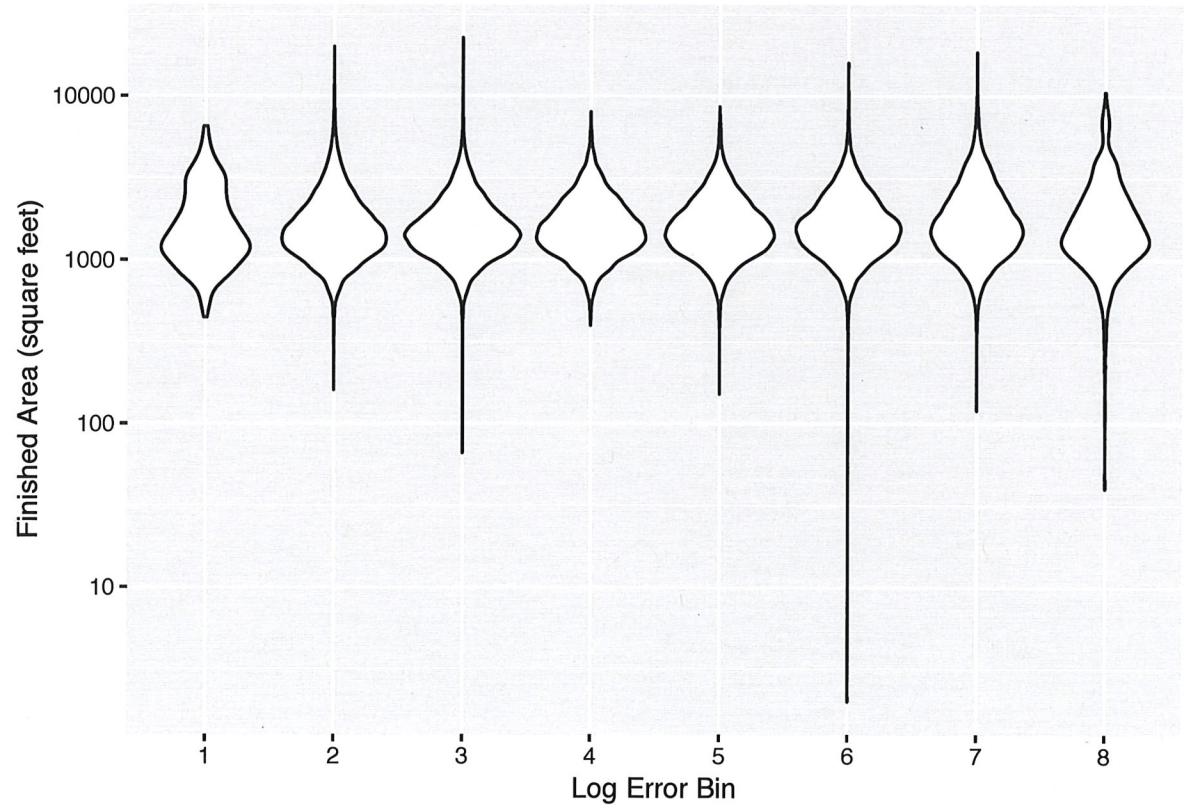


Exercise: Interpret the plots on the previous pages.

No clear pattern when comparing with bedroom count, but some evidence of larger errors when room count grows

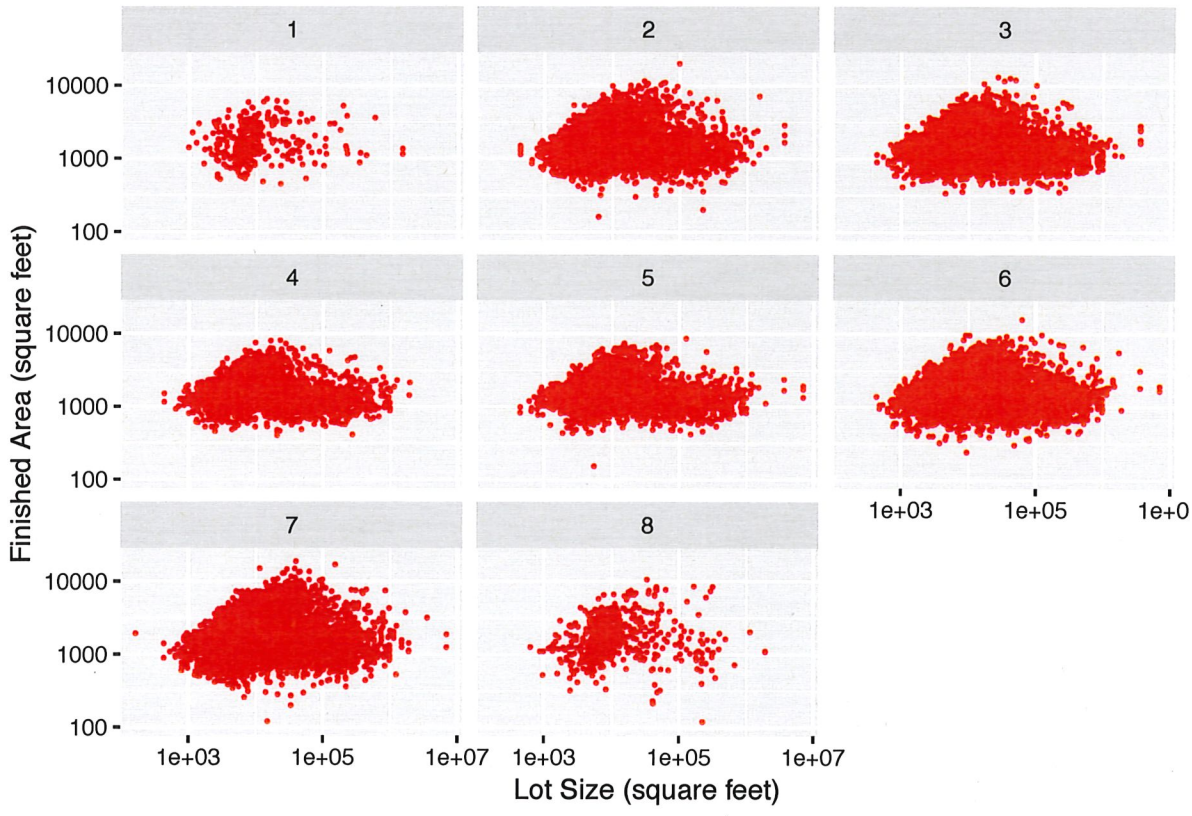
Violin plots are useful for comparing with a continuous predictor:

```
> ggplot(trainmerged, aes(x=logerrorbin,
+        y=calculatedfinishedsquarefeet)) +
+   geom_violin() + scale_y_log10() +
+   labs(x="Log Error Bin", y="Finished Area (square feet)")
Warning: Removed 661 rows containing non-finite values (stat_ydensity).
```

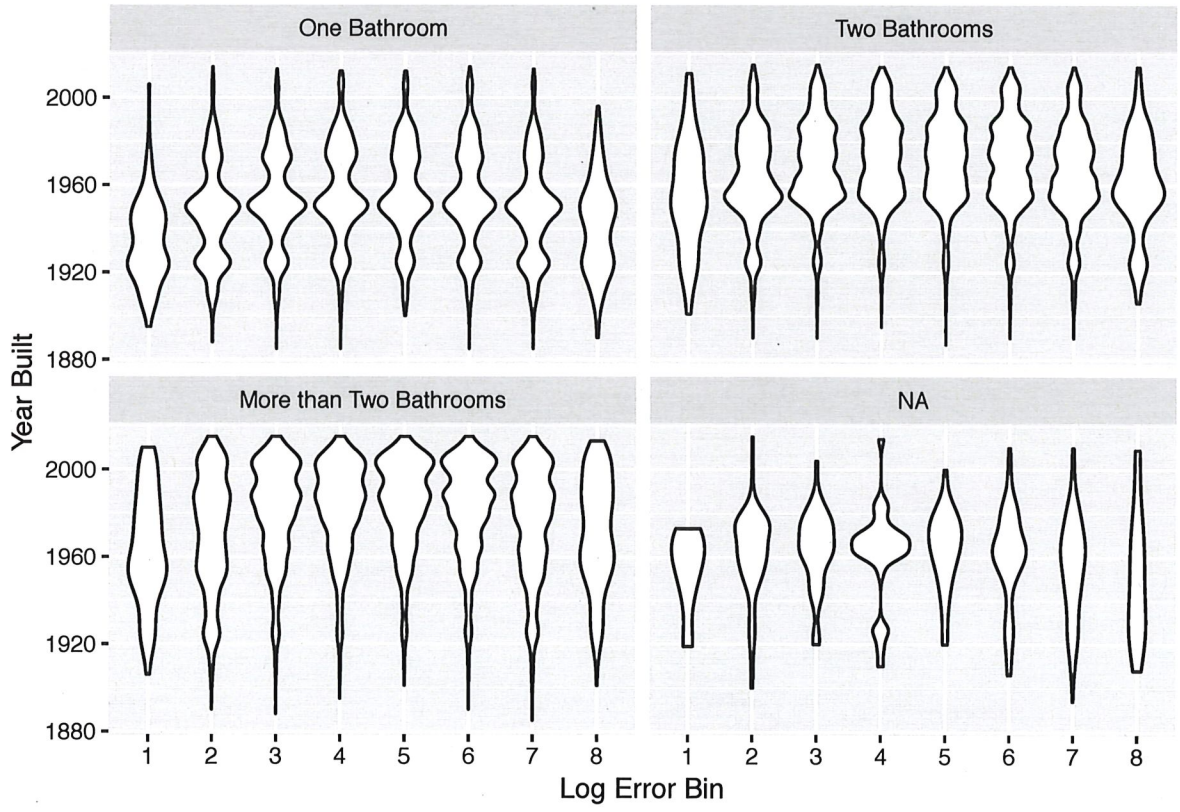
Testing for higher-order interactions can be fruitful, but this is where domain expertise starts to become very useful (or a great deal of patience), as there are many potential combinations to try.

```
> ggplot(trainmerged, aes(x=lotsizesquarefeet,
+                         y=calculatedfinishedsquarefeet)) +
+   geom_point(size = 0.3, color="red") +
+   scale_x_log10() + scale_y_log10(limits=c(100,22000)) +
+   facet_wrap(~logerrorbin) +
+   labs(x="Lot Size (square feet)",
+        y="Finished Area (square feet)")
Warning: Removed 10487 rows containing missing values (geom_point).
```

Here is an example involving one categorical and one continuous variable:

```
> trainmerged$bathcut = cut(trainmerged$fullbathcnt,
+         c(0,1,2,Inf))
> levels(trainmerged$bathcut) =
+   c("One Bathroom", "Two Bathrooms", "More than Two Bathrooms")
> ggplot(trainmerged, aes(x=factor(logerrorbin),
+                         y=yearbuilt)) +
+   geom_violin() +
+   facet_wrap(~bathcut) +
+   labs(x="Log Error Bin",
+        y="Year Built")
Warning: Removed 756 rows containing non-finite values (stat_ydensity).
```

Part 7: Smoothing

Summary

Smoothing refers generally to procedures that attempt to extract underlying signal from noise in a nonparametric manner.

The term "nonparametric" means the use of a statistical model that does not make restrictive assumptions. The data are allowed to "speak for themselves" as much as possible.

Here we consider such smoothing procedures as tools for summarizing noisy data.

Basics of Density Estimation

Nonparametric density estimation refers to techniques that estimate the distribution for a variable, without assuming any parametric form. Examples of parametric forms include the normal distribution, the exponential distribution, etc.

Histograms are a simple example of a nonparametric density estimator. While these are useful, they suffer from limitations, namely the arbitrariness of the binning, and discontinuous ("jagged") nature of the estimate of a distribution that is typically assumed to be smooth. Smooth estimators also have statistical efficiency advantages over histograms.

The standard nonparametric approach to density estimation is the kernel density estimator. This estimate is constructed by summing a smooth kernel function centered at each of the observed data points.

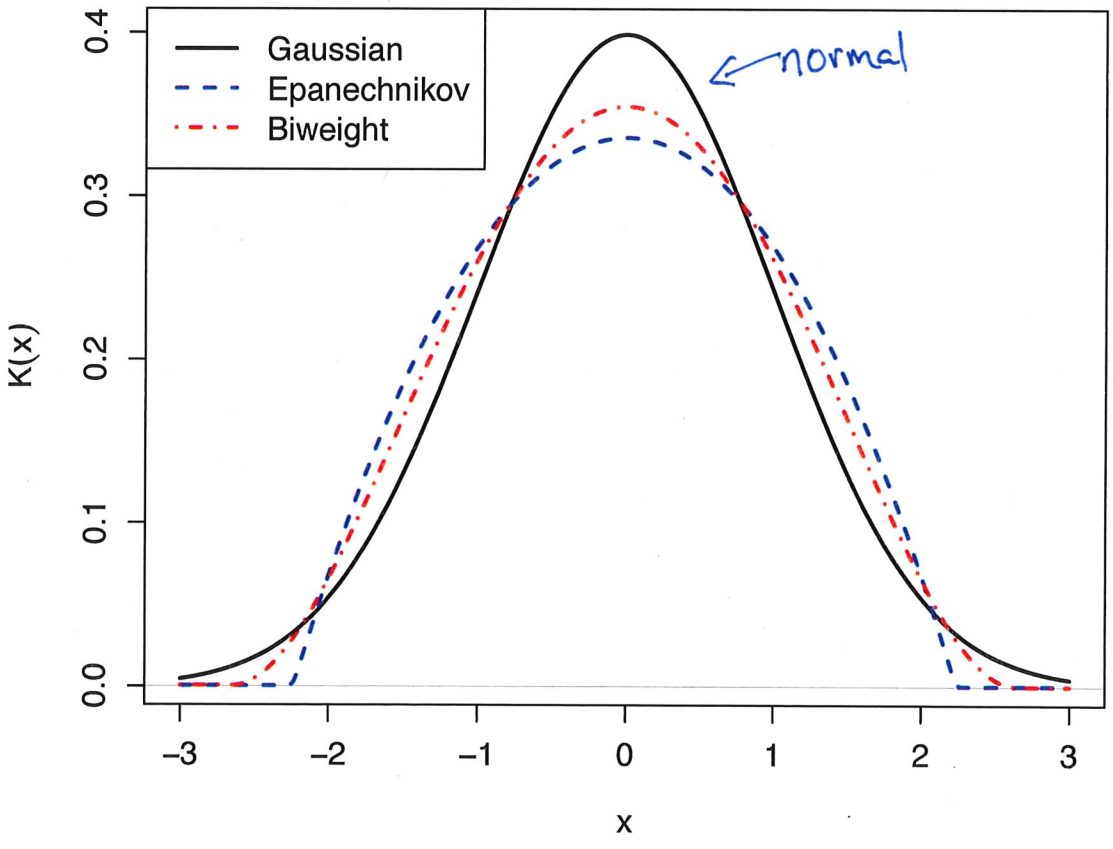
Formally, we write:

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right)$$

$\int \hat{f}_h(x)\,dx = 1$

where $K(\cdot)$ is the kernel function and h is the smoothing parameter or bandwidth. The sample is $x_1, x_2, \ldots, x_n$.

The kernel function is itself a density, almost always taken to be a smooth density peaked at zero, and symetric around zero. Three standard examples are shown on the next page.

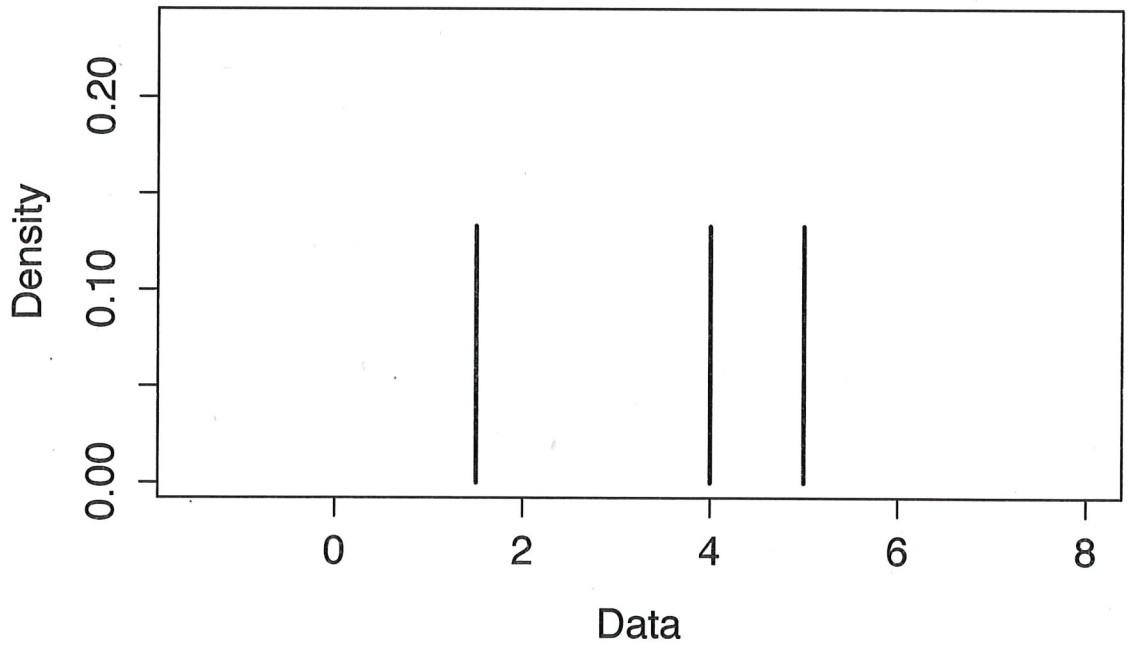


The choice of the kernel is not too influential on the shape of the density estimate. The bandwidth h is influential, however.

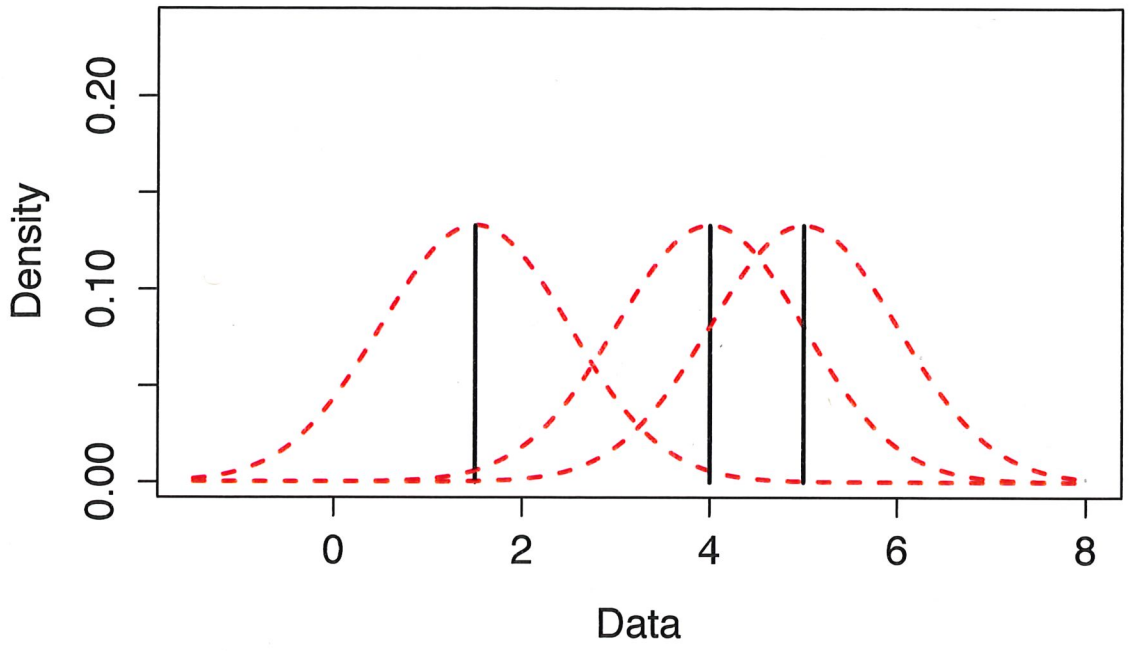
Larger values of h result in a density estimate that is smoother. Smaller values of h produce an estimate that is "rough" and "wiggly."

As a very simple example to illustrate the concept, imagine my data set consisted of three observations: 1.5, 4, and 5. The next three slides illustrate the steps going from the raw data to the final estimate. The subsequent slides show the effect of varying the bandwidth.

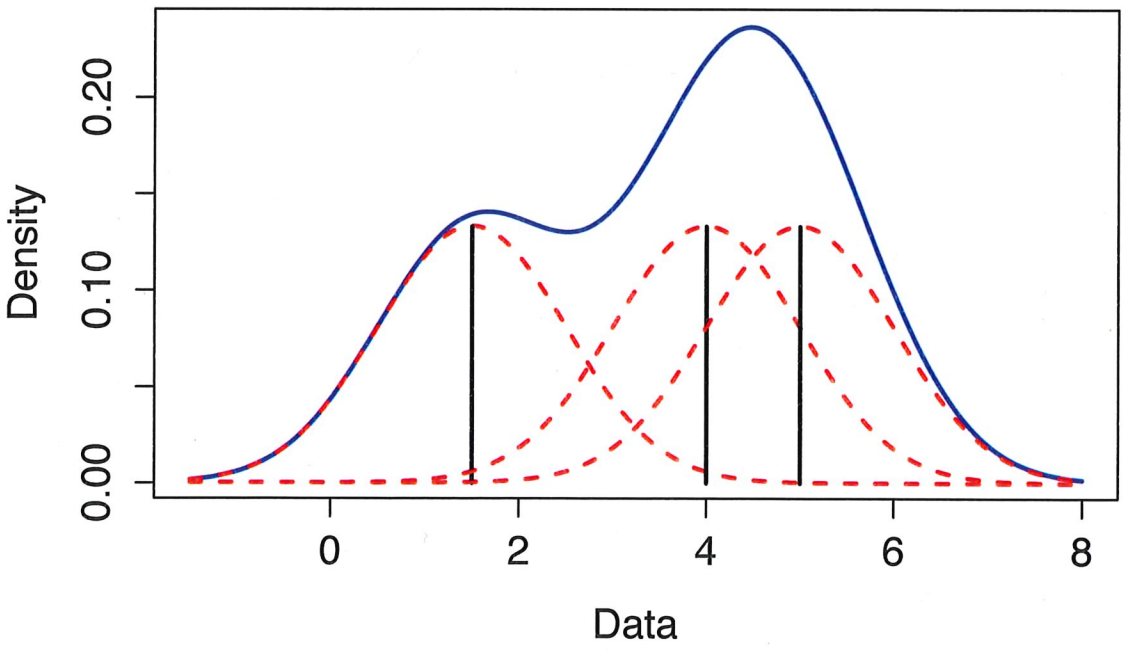
The positions of the three observations:



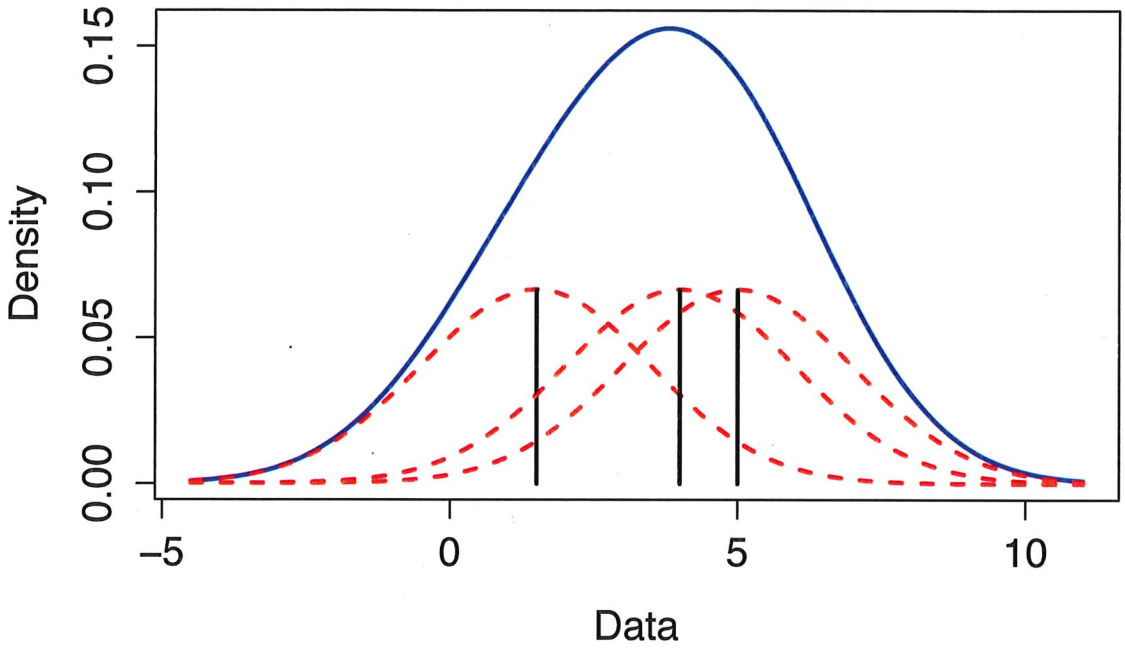
The gaussian kernel with $h = 1$ shown centered at each observation:



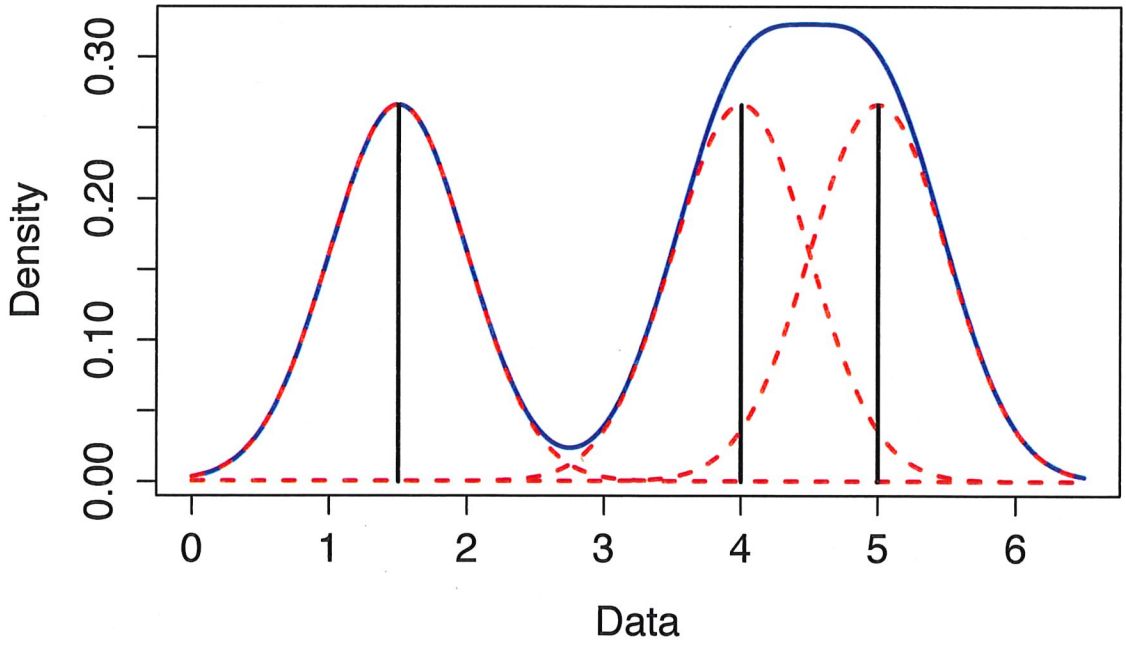
These kernels are summed to create the final estimate:



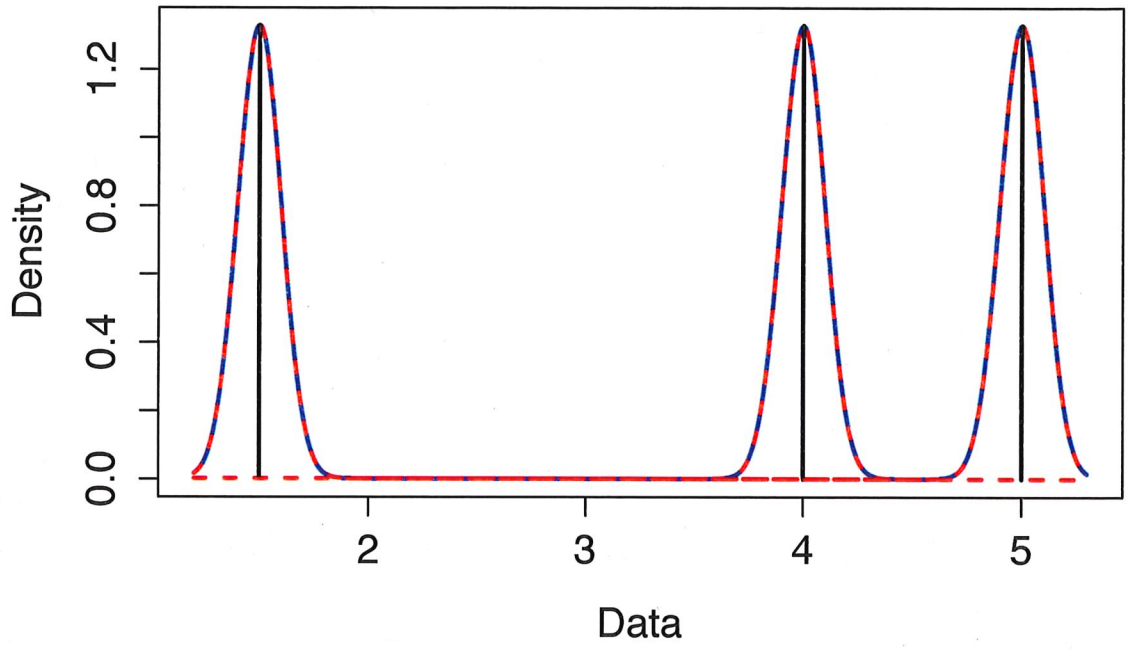
Compare this with the case where $h = 2$:



And when $h = 0.5$:



Or when $h = 0.1$:



Exercise: What is the relationship between using a Gaussian kernel and making an assumption that a sample is drawn from a Gaussian (normal) distribution?

NONE.

Assuming a normal dist. for the sample is very different from using Gaussian kernel

We use the Gaussian because of its appealing "shape", smooth, symmetric around zero.

Rigorous ways of choosing h do exist. See Jones, Marron, and Sheather (1996) for an overview.

Briefly, the theory is built on the assumption that the available sample $x_1, x_2, \ldots, x_n$ are drawn i.i.d. from some population with density $f(\cdot)$. The criterion that one seeks to minimize is the mean integrated squared error:

$$\text{MISE} = E\left[\int \left(\hat{f}_h(x) - f(x)\right)^2 dx\right].$$

As is often the case, consideration is made of the asymptotic behavior of the criterion as n increases. One can show that the the asymptotic mean integrated squared error is

$$\text{AMISE} = \frac{R(K)}{nh} + h^4 R(f'') \left(\frac{1}{2}\int x^2 K(x)\, dx\right)^2$$

where

$$R(\phi) = \int \phi^2(x)\, dx$$

variance

bias in estimator

bias/variance tradeoff

The optimal choice for h can then be shown to be

$$h = \left[\frac{R(K)}{nR(f'')\left(x^2 K(x)\right)^2}\right]^{1/5}$$

The Sheather-Jones (S-J) approach to finding the bandwidth is to find the h which solves

$$h = \left[\frac{R(K)}{nR(\hat{f}''_{g(h)})\left(x^2 K(x)\right)^2}\right]^{1/5}$$

where $g(h)$ is some function of h which accounts for the fact that the optimal choice of bandwidth for estimating f'' is not the same as the optimal choice for estimating f.

The choice of h can also be dictated by less theoretical considerations.

In particular, if the motivation is to achieve a summary of a distribution via smoothing the distribution to a particular scale.

One could, for example, smooth a distribution using multiple bandwidth choices, and then using the resulting smooth estimates as input into a model. Formal procedures could then be used to determine how useful each "scale" is to the problem at hand, i.e., a prediction challenge.

This idea will be explored through examples later.

Kernel Density Estimation in R

The basic function for kernel density estimation in R is `density()`.

The argument `kernel` specifies the kernel function; the default is `"gaussian"`.

The bandwidth is provided using the argument `bw`. The user can specify either a number, or specify a method for determining the bandwidth. For example, the function `bw="SJ"` calculates the S-J bandwidth.

The argument `adjust` can be useful when attempting to try multiple scales. The bandwidth utilized is actually equal to `adjust` times the value of `bw`. This makes it easy to fit with, e.g., double the S-J bandwidth, half the S-J bandwidth, etc.

Exercise: Try the following code, and discuss the structure of the output of `density()`. What happens as the sample size is varied?

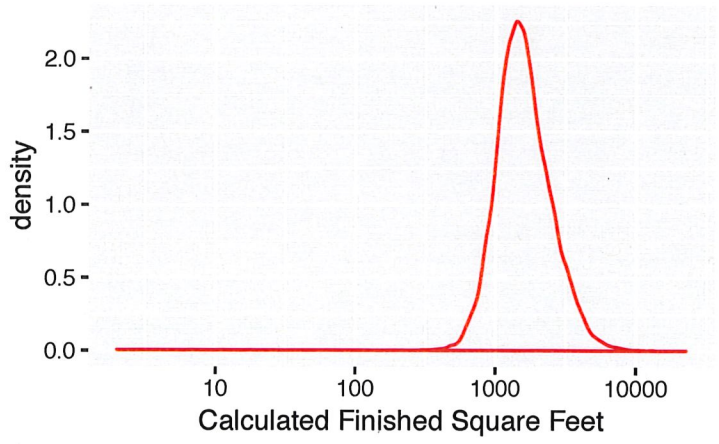
```
> x = rnorm(1000)
> densout = density(x, bw="SJ")
> plot(densout)
> curve(dnorm(x), add=T, col="red", lty=2)
```

densout$x is the grid of values at which the density is estimated

densout$y is the estimated density

There is a function `geom_density()` as part of `ggplot`. It accepts the same arguments as does `density()`.

```
> ggplot(trainmerged, aes(x=calculatedfinishedsquarefeet)) +
+     geom_density(bw="SJ", color="red") +
+     labs(x="Calculated Finished Square Feet") +
+     scale_x_log10()
```



Exercise: How does one interpret the vertical scale "Density" on the preceding plot?

pdf's are constructed so that they integrate to 1 $\int_{-\infty}^{\infty} f(x)\,dx = 1$

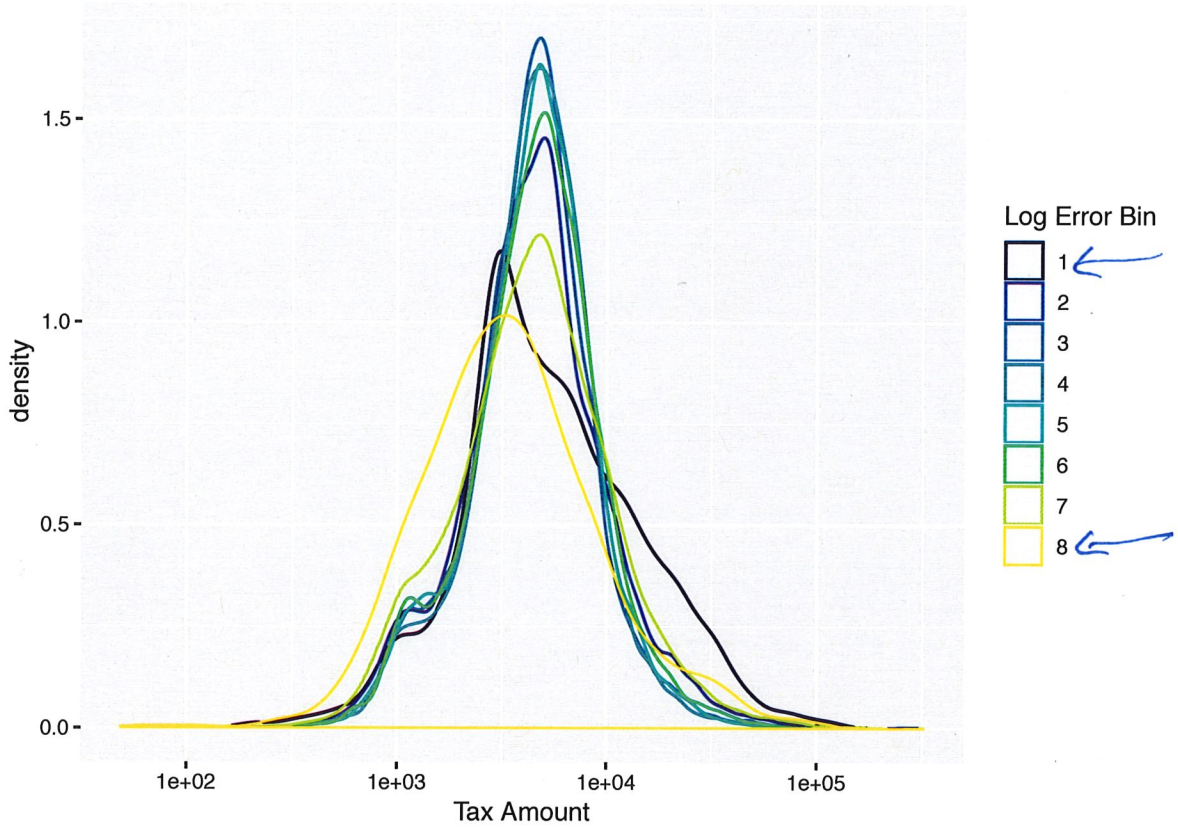
Same holds for the KDE: $\int_{-\infty}^{\infty} \hat{f}(x)\,dx = 1$

Insights from Density Estimates

As stated above, a primary motivation for smoothing in general, and non-parametric density estimation in particular, is to work to separate real features (the "signal") from the uninformative randomness (the "noise").

Consider how the next plot presents changes in the distribution of the amount of property tax over the different log error bins. This builds on our previous Zillow example.

```
> ggplot(trainmerged, aes(x=taxamount, color=logerrorbin)) +
+     geom_density(bw="SJ") +
+     labs(x="Tax Amount", color="Log Error Bin") +
+     scale_x_log10()
```



Exercise: What conclusion(s) could you draw from the previous plot?

The instances where the errors are largest (neg. and posi) show distributions with larger spreads than those with small absolute errors

A kernel density estimate is often utilized to determine an appropriate "named" parametric distribution, if it exists.

As an example, use package `quantmod` to obtain the daily data for Kellogg's (K) from 2010 through 2016:

```
> Kellogg = getSymbols("K",
+   from="2010-1-1",to="2016-12-31", auto.assign=F)
```

Then calculate the log daily returns:

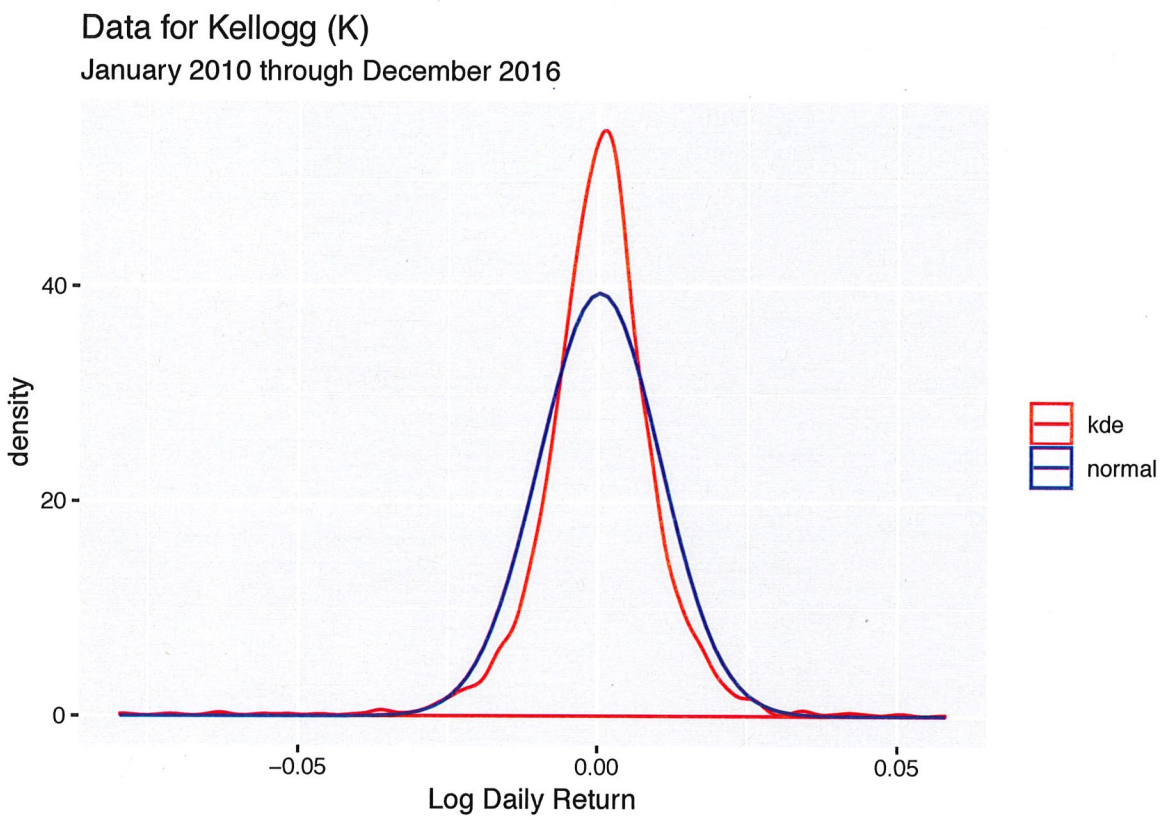
adjusted closing price

```
> ldrK = data.frame(dailyReturn(Ad(Kellogg),
+                               type="log"))
```

The function `Ad()` extracts the "adjusted closing price" column from the data set.

Consider the following plot which compares the kernel density estimate with the normal distribution:

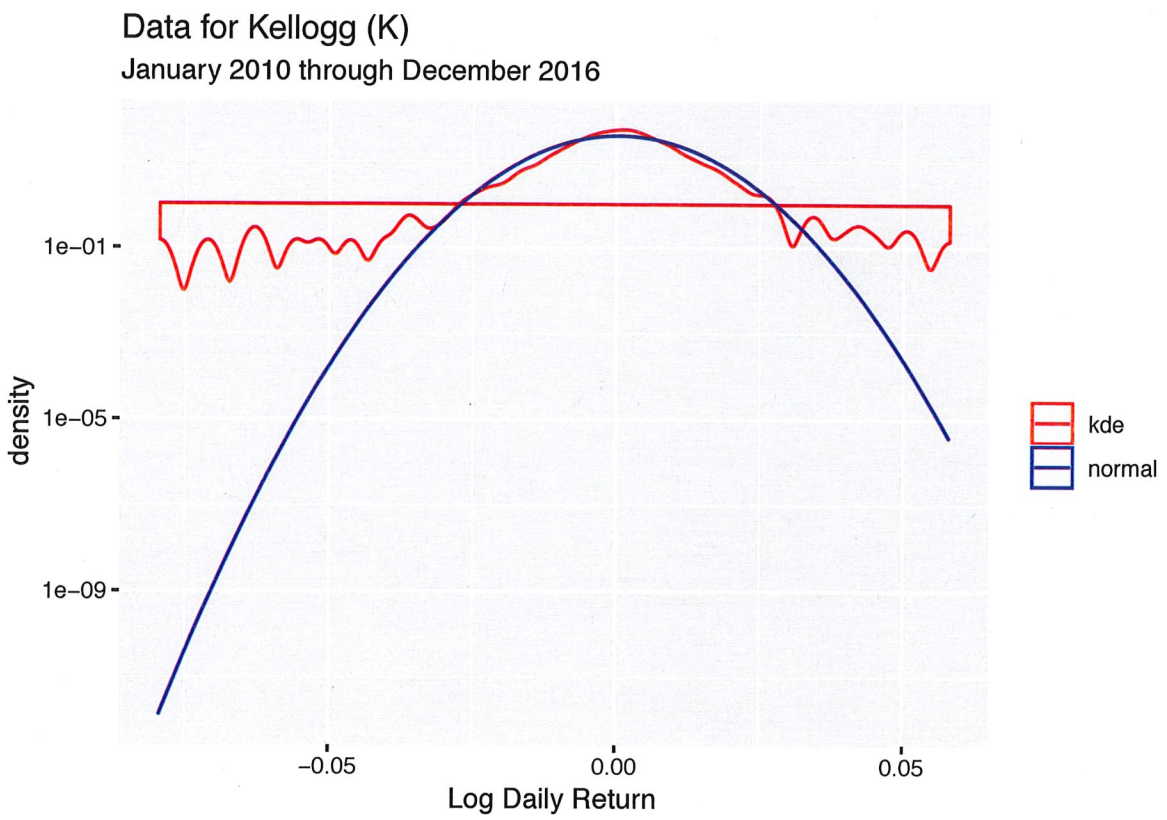
```
> ggplot(ldrK,aes(x=daily.returns)) +
+   geom_density(bw="SJ",aes(color="kde")) +
+   stat_function(fun=dnorm, aes(color="normal"),
+       args=list(mean=mean(ldrK$daily.returns),
+                 sd=sd(ldrK$daily.returns))) +
+   scale_color_manual(name="",
+             values=c("kde"="red","normal"="blue")) +
+   labs(x="Log Daily Return",title="Data for Kellogg (K)",
+        subtitle="January 2010 through December 2016")
```



1

1 The plot is not as informative as one on the log scale:

```
> ggplot(ldrK,aes(x=daily.returns)) +
+   geom_density(bw="SJ",aes(color="kde")) +
+   stat_function(fun=dnorm, aes(color="normal"),
+       args=list(mean=mean(ldrK$daily.returns),
+                 sd=sd(ldrK$daily.returns))) +
+   scale_color_manual(name="",
+              values=c("kde"="red","normal"="blue")) +
+   labs(x="Log Daily Return",title="Data for Kellogg (K)",
+        subtitle="January 2010 through December 2016") +
+   scale_y_log10()
```



Exercise: Comment on the appropriateness of the normal approximation to log returns in this case.

It's clear from the plot on the log scale that the normal dist. does not assign sufficient prob. to the tails.

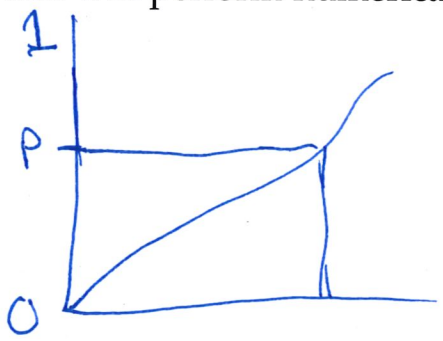
The KDE reveals the heavy tails of the log return distribution.

Converting to the CDF and Percentiles

Smooth density estimates can be transformed into an estimate of the cumulative distribution function (CDF) and its inverse (i.e., percentiles).

A useful function in R to achieve this is `approxfun()`. It takes `x` and `y` vectors and returns a function which linearly interpolates the given values. Note that the output of `approxfun()` is itself a function.

Also, there is a function `integrate()` that will perform numerical integration.



Consider the following function `kCDF()`:

```
> kCDF = function(x, res=100, ...)
+ {
+     holddens = density(x,...)
+     interpdens = approxfun(holddens$x, holddens$y,
+                            yleft=0, yright=0)
+     xseq = seq(min(holddens$x),max(holddens$x),length=res)
+     holdout = numeric(res)
+     for(i in 1:res)
+     {
+        holdout[i] = integrate(interpdens, lower=min(holddens$x),
+                     upper=xseq[i], stop.on.error=FALSE)$value
+     }
+     CDF = approxfun(xseq, holdout, yleft=0, yright=1)
+     invCDF = approxfun(holdout, xseq, yleft=NA, yright=NA)
+     list(CDF=CDF, invCDF=invCDF)
+ }
```

$$F_X(x) = \int_{-\infty}^{x} f(u)\,du$$

Exercise: Discuss what the function `kCDF()` will do.

For this example, let's return to the market structure data that we considered in Parts 4 and 5:

```
> fulldata = read.table("q1_2017_all.csv", sep=",", header=T, quote="")
> fulldata$Date = as.Date(as.character(fulldata$Date), format="%Y%m%d")
> fulldata = fulldata[!duplicated(fulldata[,c(1,3)], fromLast=TRUE),]
> fulldata$McapRank = factor(fulldata$McapRank, ordered=TRUE)
> fulldata$TurnRank = factor(fulldata$TurnRank, ordered=TRUE)
> fulldata$VolatilityRank = factor(fulldata$VolatilityRank, ordered=TRUE)
> fulldata$PriceRank = factor(fulldata$PriceRank, ordered=TRUE)
```

We will consider the variable `Cancels`. Split this variable up by the ticker symbol, and exclude those with fewer than 5 observations:

```
> cancelbyticker = split(fulldata$Cancels,fulldata$Ticker)
> cancelbyticker =
+   cancelbyticker[sapply(cancelbyticker,length)>=5]
```